

Neural ODEs

Differentiable Physical Modelling and Machine Learning for Sound Synthesis

Rodrigo Diaz

2026-06-04

Overview

- Recap: differentiable simulators, losses, and constrained parameter learning
- Revisiting continuous-time state-space models through differentiable solvers
- Fixed-step ODE solvers as differentiable program blocks
- Neural ODEs as learnable continuous-time dynamics on the audio grid
- Backpropagating through trajectories: direct AD, adjoints, and practical caveats
- Hybrid physical-neural models for modal sound synthesis
- Mini task: learn a missing force or residual from modal trajectories
- Stability, energy, stiffness, and solver choices for audio-rate systems
- Bridge to next week: physics-informed losses and residual constraints

Recap: Differentiable Audio Models

The Week 4 Loop

Last week we wrote a synthesiser as a differentiable function:

$$\theta \longrightarrow \hat{y} = g(\theta)$$

Then we chose a loss:

$$J(\theta) = \ell(\hat{y}, y)$$

And used automatic differentiation:

$$\nabla_{\theta} J$$

to update the parameters.

What Was Learnable?

In the simple sinusoidal example:

$$\hat{y}(t) = Ae^{-\gamma t} \sin(2\pi f t + \varphi)$$

the learnable quantities were:

$$\theta = \{A, \gamma, f, \varphi\}$$

In the modal case, we could learn:

- modal frequencies ω_μ
- modal damping rates γ_μ
- initial modal amplitudes
- readout weights
- physical parameters that generate the modal spectrum

What Was Still Fixed?

Even when the parameters were learnable, the form of the model was fixed.

For a linear modal system:

$$\dot{\mathbf{q}} = \mathbf{A}\tilde{\mathbf{q}}$$

For the nonlinear string:

$$\ddot{\mathbf{q}} = -2\Gamma\dot{\mathbf{q}} - \Omega^2\mathbf{q} - \mathbf{f}_{\text{nl}}(\mathbf{q})$$

The optimiser could change numbers inside the equations, but not the equation family itself.

Continuous-Time State-Space Models

The State-Space View

The common form underneath the course is:

$$\dot{\mathbf{h}}(t) = \mathbf{F}(\mathbf{h}(t), t, f(t); \theta)$$

where:

- $\mathbf{h}(t)$ is the state
- $f(t)$ is the external force or excitation signal
- θ contains parameters
- $\mathbf{F}$ is the vector field: it says how the state changes

The output is usually a readout:

$$\hat{y}(t) = \mathbf{C}\mathbf{h}(t)$$

This keeps the notation from Weeks 1-2: $f(t)$ is the driving signal, and $\mathbf{C}$ is the output map.

From Physics to a Vector Field

For a modal second-order system:

$$\ddot{\mathbf{q}} = -2\Gamma\dot{\mathbf{q}} - \Omega^2\mathbf{q} + \mathbf{b}f(t)$$

Stack displacement and velocity directly:

$$\frac{d}{dt} \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ -\Omega^2 & -2\Gamma \end{bmatrix} \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \mathbf{b} \end{bmatrix} f(t)$$

This is a continuous-time state-space vector field with a fixed physical matrix.

The Simulator

The computer never stores the whole continuous trajectory.

It solves the initial value problem:

$$\mathbf{h}(t_0) = \mathbf{h}^0, \quad \dot{\mathbf{h}} = \mathbf{F}(\mathbf{h}, t, f(t); \theta)$$

and returns samples:

$$\mathbf{h}^0, \mathbf{h}^1, \dots, \mathbf{h}^N \quad \text{where} \quad \mathbf{h}^n \approx \mathbf{h}(t^n)$$

The discrete recursion is an approximation to the continuous-time model.

Revisiting the Pipeline

Week 4:

$$\theta \longrightarrow \text{physical simulator} \longrightarrow \hat{y} \longrightarrow \ell(\hat{y}, y) \longrightarrow \nabla_{\theta} J$$

Week 5:

$$\text{physical dynamics} + \mathbf{f}_{\text{nl},\theta} \longrightarrow \text{fixed-step solver} \longrightarrow \hat{y}$$

The neural part lives inside the dynamics; the solver still turns those dynamics into an audio-rate recurrence.

ODE Solvers as Differentiable Blocks

The Solver Is Part of the Model

A continuous-time model is not executable until we choose a solver:

$\mathbf{F}$ and a time-stepping rule

In this lecture we focus on **fixed-step solvers** on the audio grid:

$$t^n = n\Delta t, \quad n = 0, \dots, N$$

The solver turns a continuous-time system into a recurrence:

$$\mathbf{h}^{n+1} = \Phi_{\Delta t}(\mathbf{h}^n, f^n)$$

For audio synthesis, the fixed grid matters because the model must return the whole trajectory.

Then the audio readout is:

$$\hat{y}^n = \mathbf{C}\mathbf{h}^n$$

Change the solver and you change the recurrence.

Discretising a Vector Field

Start from any first-order ODE:

$$\frac{d}{dt}\mathbf{h}(t) = \mathbf{F}(\mathbf{h}(t), t, f(t))$$

On the fixed grid, a solver replaces the time derivative by a difference rule:

$$\delta_t^{(\text{solver})} \mathbf{h}^n \approx \mathbf{F}(\cdot)$$

Here $\mathbf{F}$ is agnostic:

- physical vector field
- neural vector field
- hybrid physical-neural vector field

The choice of $\delta_t^{(\text{solver})}$ determines the discrete recurrence.

Fixed-Step Solver Examples

Different solvers choose different discrete forms for $\delta_t^{(\text{solver})}$.

Forward Euler

$$\delta_{t,+} \mathbf{h}^n := \frac{\mathbf{h}^{n+1} - \mathbf{h}^n}{\Delta t} = \mathbf{F}(\mathbf{h}^n, t^n, f^n)$$

Centred first difference / leapfrog

$$\delta_t \mathbf{h}^n := \frac{\mathbf{h}^{n+1} - \mathbf{h}^{n-1}}{2\Delta t} = \mathbf{F}(\mathbf{h}^n, t^n, f^n)$$

Other fixed-step solvers, such as Runge-Kutta methods, use additional evaluations of $\mathbf{F}$ inside each audio-grid step.

For Oscillators: Use the Structure

For the modal systems we care about, the dynamics are naturally second order:

$$\ddot{\mathbf{q}} = \text{acceleration}$$

So instead of applying a generic first-order solver blindly, we use a scheme designed for second-order oscillators.

Why:

- it keeps the displacement/velocity structure explicit
- it has better long-time behaviour for oscillatory systems
- it matches the centred finite-difference schemes from Weeks 1–2

Start With a Second-Order Oscillator

For one modal oscillator, or a diagonal bank of modes:

$$\ddot{\mathbf{q}} = -2\Gamma\dot{\mathbf{q}} - \Omega^2\mathbf{q} + \mathbf{b}f(t)$$

Generic ODE solvers evolve first-order systems. As in Week 1, stack displacement and velocity:

$$\mathbf{h} = \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix}$$

Then the second-order oscillator becomes:

$$\dot{\mathbf{h}} = \begin{bmatrix} \dot{\mathbf{q}} \\ -2\Gamma\dot{\mathbf{q}} - \Omega^2\mathbf{q} + \mathbf{b}f(t) \end{bmatrix}$$

This first-order vector field is what a generic ODE solver advances.

Finite Difference Operators

From Week 1, the centred time-difference operators are:

$$\delta_t q^n = \frac{q^{n+1} - q^{n-1}}{2\Delta t}$$

$$\delta_{tt} q^n = \frac{q^{n+1} - 2q^n + q^{n-1}}{\Delta t^2}$$

The first approximates velocity; the second approximates acceleration.

Störmer-Verlet

Start from the damped second-order oscillator:

$$\ddot{\mathbf{q}} = -2\Gamma\dot{\mathbf{q}} - \Omega^2\mathbf{q} + \mathbf{b}f(t)$$

The acceleration is not just a function of $\mathbf{q}$: the damping term depends on velocity.

Apply the centred operators directly to the second-order equation:

$$\delta_{tt}\mathbf{q}^n = -2\Gamma\delta_t\mathbf{q}^n - \Omega^2\mathbf{q}^n + \mathbf{b}f^n$$

Here $\delta_t\mathbf{q}^n$ is a velocity estimate from displacement samples, not an independent velocity state.

Expand the Differences

Expand the finite-difference operators:

$$\frac{\mathbf{q}^{n+1} - 2\mathbf{q}^n + \mathbf{q}^{n-1}}{\Delta t^2} = -2\Gamma \frac{\mathbf{q}^{n+1} - \mathbf{q}^{n-1}}{2\Delta t} - \Omega^2 \mathbf{q}^n + \mathbf{b}f^n$$

Because the damping term uses a centred velocity estimate, $\mathbf{q}^{n+1}$ appears on both sides.
The next slide solves this linear relation.

Solve for the Next Sample

Starting from the expanded equation:

$$\frac{\mathbf{q}^{n+1} - 2\mathbf{q}^n + \mathbf{q}^{n-1}}{\Delta t^2} = -2\mathbf{\Gamma} \frac{\mathbf{q}^{n+1} - \mathbf{q}^{n-1}}{2\Delta t} - \mathbf{\Omega}^2 \mathbf{q}^n + \mathbf{b}f^n$$

Collect the unknown $\mathbf{q}^{n+1}$ terms:

$$(\mathbf{I} + \Delta t \mathbf{\Gamma}) \mathbf{q}^{n+1} = (2\mathbf{I} - \Delta t^2 \mathbf{\Omega}^2) \mathbf{q}^n - (\mathbf{I} - \Delta t \mathbf{\Gamma}) \mathbf{q}^{n-1} + \Delta t^2 \mathbf{b}f^n$$

With $\mathbf{R} = (\mathbf{I} + \Delta t \mathbf{\Gamma})^{-1}$:

$$\boxed{\mathbf{q}^{n+1} = \mathbf{R}(2\mathbf{I} - \Delta t^2 \mathbf{\Omega}^2) \mathbf{q}^n - \mathbf{R}(\mathbf{I} - \Delta t \mathbf{\Gamma}) \mathbf{q}^{n-1} + \Delta t^2 \mathbf{R} \mathbf{b}f^n}$$

This is the same two-step modal recursion from Week 2. In modal coordinates, $\mathbf{\Gamma}$ is diagonal, so $\mathbf{R}$ is elementwise.

Two-Step State-Space Matrix

Stack two consecutive displacement samples:

$$\mathbf{h}^n = \begin{bmatrix} \mathbf{q}^n \\ \mathbf{q}^{n-1} \end{bmatrix}, \quad \mathbf{h}^{n+1} = \begin{bmatrix} \mathbf{q}^{n+1} \\ \mathbf{q}^n \end{bmatrix}$$

Then the centred-difference scheme is a discrete state-space recursion:

$$\mathbf{h}^{n+1} = \mathbf{A}_\Delta \mathbf{h}^n + \mathbf{b}_\Delta f^n$$

$$\mathbf{A}_\Delta = \begin{bmatrix} \mathbf{R}(2\mathbf{I} - \Delta t^2 \Omega^2) & -\mathbf{R}(\mathbf{I} - \Delta t \Gamma) \\ \mathbf{I} & \mathbf{0} \end{bmatrix}, \quad \mathbf{b}_\Delta = \begin{bmatrix} \Delta t^2 \mathbf{R} \mathbf{b} \\ \mathbf{0} \end{bmatrix}$$

For a displacement readout:

$$\hat{\mathbf{y}}^n = \mathbf{C}_\Delta \mathbf{h}^n, \quad \mathbf{C}_\Delta = [\mathbf{C}_q \quad \mathbf{0}]$$

If We Want Velocity as a State

Start again from the centred-difference formulation:

$$\delta_{tt}\mathbf{q}^n = -2\Gamma \delta_t \mathbf{q}^n - \Omega^2 \mathbf{q}^n + \mathbf{b}_f^n$$

The two-step version represents both $\delta_t \mathbf{q}^n$ and $\delta_{tt} \mathbf{q}^n$ using only displacement samples.

The centred velocity at time n is

$$\mathbf{v}^n := \delta_t \mathbf{q}^n = \frac{\mathbf{q}^{n+1} - \mathbf{q}^{n-1}}{2\Delta t}.$$

To keep velocity as a state, factor those same centred operators through half-step velocities:

$$\mathbf{v}^{n+\frac{1}{2}} = \frac{\mathbf{q}^{n+1} - \mathbf{q}^n}{\Delta t}, \quad \mathbf{v}^{n-\frac{1}{2}} = \frac{\mathbf{q}^n - \mathbf{q}^{n-1}}{\Delta t}$$

These half-step velocities are intermediate quantities. The stored state will be the integer-time pair $[\mathbf{q}^n, \mathbf{v}^n]^\top$.

Velocity Verlet Half-Step

If velocity is available at the integer time n , construct the half-step velocity with a half kick:

$$\mathbf{v}^{n+\frac{1}{2}} = \mathbf{v}^n + \frac{\Delta t}{2} \mathbf{a}^n$$

The integer time position and integer time velocity are related by a half step:

$$\begin{aligned}\mathbf{q}^{n+1} &= \mathbf{q}^n + \Delta t \mathbf{v}^{n+\frac{1}{2}} \\ \mathbf{v}^{n+1} &= \mathbf{v}^{n+\frac{1}{2}} + \frac{\Delta t}{2} \mathbf{a}^{n+1}\end{aligned}$$

For the damped oscillator at time n and $n + 1$:

$$\begin{aligned}\mathbf{a}^n &= -2\Gamma \mathbf{v}^n - \Omega^2 \mathbf{q}^n + \mathbf{b} f^n \\ \mathbf{a}^{n+1} &= -2\Gamma \mathbf{v}^{n+1} - \Omega^2 \mathbf{q}^{n+1} + \mathbf{b} f^{n+1}\end{aligned}$$

Because $\mathbf{a}^{n+1}$ depends on $\mathbf{v}^{n+1}$, the final half kick is implicit.

Solving the Final Half Kick

Substitute the damped acceleration into the final half kick:

$$\mathbf{v}^{n+1} = \mathbf{v}^{n+\frac{1}{2}} + \frac{\Delta t}{2} \left(-2\mathbf{\Gamma}\mathbf{v}^{n+1} - \mathbf{\Omega}^2\mathbf{q}^{n+1} + \mathbf{b}f^{n+1} \right)$$

Move the unknown velocity terms to the left:

$$\mathbf{v}^{n+1} + \Delta t \mathbf{\Gamma}\mathbf{v}^{n+1} = \mathbf{v}^{n+\frac{1}{2}} - \frac{\Delta t}{2} \mathbf{\Omega}^2\mathbf{q}^{n+1} + \frac{\Delta t}{2} \mathbf{b}f^{n+1}$$

Factor $\mathbf{v}^{n+1}$:

$$(\mathbf{I} + \Delta t \mathbf{\Gamma})\mathbf{v}^{n+1} = \mathbf{v}^{n+\frac{1}{2}} - \frac{\Delta t}{2} \mathbf{\Omega}^2\mathbf{q}^{n+1} + \frac{\Delta t}{2} \mathbf{b}f^{n+1}$$

The $\mathbf{I}$ appears because $\mathbf{v}^{n+1} = \mathbf{I}\mathbf{v}^{n+1}$.

Velocity Verlet State-Space Form

Define the integer-time state and helper matrices:

$$\mathbf{h}^n = \begin{bmatrix} \mathbf{q}^n \\ \mathbf{v}^n \end{bmatrix}, \quad \mathbf{P} = \mathbf{I} - \Delta t \mathbf{\Gamma}, \quad \mathbf{Q} = \mathbf{I} - \frac{\Delta t^2}{2} \mathbf{\Omega}^2, \quad \mathbf{R} = (\mathbf{I} + \Delta t \mathbf{\Gamma})^{-1}$$

Then velocity Verlet gives a one-step recursion:

$$\begin{aligned} \mathbf{h}^{n+1} &= \mathbf{A}_{\text{vv}} \mathbf{h}^n + \mathbf{b}_0 f^n + \mathbf{b}_1 f^{n+1} \\ \mathbf{A}_{\text{vv}} &= \begin{bmatrix} \mathbf{Q} & \Delta t \mathbf{P} \\ -\frac{\Delta t}{2} \mathbf{R} \mathbf{\Omega}^2 (\mathbf{I} + \mathbf{Q}) & \mathbf{R} \left(\mathbf{P} - \frac{\Delta t^2}{2} \mathbf{\Omega}^2 \mathbf{P} \right) \end{bmatrix} \\ \mathbf{b}_0 &= \begin{bmatrix} \frac{\Delta t^2}{2} \mathbf{b} \\ \mathbf{R} \left(\frac{\Delta t}{2} \mathbf{b} - \frac{\Delta t^3}{4} \mathbf{\Omega}^2 \mathbf{b} \right) \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} \mathbf{0} \\ \frac{\Delta t}{2} \mathbf{R} \mathbf{b} \end{bmatrix} \end{aligned}$$

The extra f^{n+1} term comes from the final half kick.

Neural ODEs

Neural ODE: Definition

A Neural ODE parameterises the vector field with a neural network:

$$\dot{\mathbf{h}}(t) = \mathbf{F}_{\theta}(\mathbf{h}(t), t, f(t))$$

The trajectory is defined implicitly by integration:

$$\mathbf{h}(t_1) = \mathbf{h}(t_0) + \int_{t_0}^{t_1} \mathbf{F}_{\theta}(\mathbf{h}(t), t, f(t)) dt$$

The neural network does not output the next state directly.

It outputs the **instantaneous rate of change**, also called the vector field.

The solver integrates this vector field to produce the flow from $\mathbf{h}(t_0)$ to $\mathbf{h}(t_1)$.

In this course, we usually evaluate that flow with a **fixed-step solver** on the audio grid.

Neural ODE vs. Discrete RNN

Residual RNN

$$\mathbf{h}^{n+1} = \mathbf{h}^n + \Delta t \mathbf{G}_\theta(\mathbf{h}^n, f^n)$$

- learns the residual update on the grid
- has the same form as a forward Euler step
- tied to the chosen Δt
- one network call per step
- natural when the data are sampled sequences

Neural ODE

$$\dot{\mathbf{h}} = \mathbf{F}_\theta(\mathbf{h}, t, f(t))$$

Forward Euler gives:

$$\mathbf{h}^{n+1} = \mathbf{h}^n + \Delta t \mathbf{F}_\theta(\mathbf{h}^n, t^n, f^n)$$

- learns the continuous-time vector field
- the solver turns it into a recurrence
- with Euler, it resembles a residual RNN cell

Where Does the Learning Happen?

Both produce a recurrence on the audio grid. The difference is where the learnable object lives:

learn $\mathbf{G}_\theta$ or learn $\mathbf{F}_\theta$ and discretise it with $\Phi_{\Delta t}$

For audio, arbitrary-time evaluation is not the main selling point. We usually need the full trajectory on a fixed sample grid:

$$t^n = n\Delta t, \quad n = 0, \dots, N$$

Why This Fits Physical Modelling

Physical modelling already starts from:

$$\dot{\mathbf{h}} = \mathbf{F}_{\text{phys}}(\mathbf{h}, t, f(t); \phi)$$

Neural ODEs let us replace this with:

$$\dot{\mathbf{h}} = \mathbf{F}_{\theta}(\mathbf{h}, t, f(t))$$

or, more usefully for this course:

$$\dot{\mathbf{h}} = \mathbf{F}_{\text{phys}}(\mathbf{h}, t, f(t); \phi) + \mathbf{f}_{\text{nl}, \theta}(\mathbf{h}, t, f(t))$$

The network learns a residual force, not the whole instrument from scratch.

Three Modelling Choices

Pure physics

$$\dot{\mathbf{h}} = \mathbf{F}_{\text{phys}}(\mathbf{h})$$

Interpretable, structured, but incomplete.

Pure Neural

ODE

$$\dot{\mathbf{h}} = \mathbf{F}_{\theta}(\mathbf{h})$$

Flexible, but weak physical guarantees.

Hybrid

$$\dot{\mathbf{h}} = \mathbf{F}_{\text{phys}}(\mathbf{h}) + \mathbf{f}_{\text{nl},\theta}(\mathbf{h})$$

Structured baseline plus learned correction.

For sound synthesis, the hybrid case is often the most useful starting point.

Three Choices in State-Space Form

Let the known physical acceleration be:

$$\mathbf{A}_{\text{phys}} = \begin{bmatrix} 0 & \mathbf{I} \\ -\Omega^2 & -2\mathbf{\Gamma} \end{bmatrix}, \quad \mathbf{b}_{\text{phys}} = \begin{bmatrix} 0 \\ \mathbf{b} \end{bmatrix}$$

Pure physics

$$\frac{d}{dt} \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix} = \mathbf{A}_{\text{phys}} \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix} + \mathbf{b}_{\text{phys}} f(t)$$

Pure neural

$$\frac{d}{dt} \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix} = \mathbf{F}_{\theta}(\mathbf{q}, \dot{\mathbf{q}}, f(t))$$

Hybrid

$$\frac{d}{dt} \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix} = \mathbf{A}_{\text{phys}} \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix} + \mathbf{b}_{\text{phys}} f(t) +$$

What Can the Network Represent?

A residual can learn:

- unknown restoring forces
- missing damping
- mode coupling
- truncation or discretisation error
- latent excitation effects

Only what the data and loss make identifiable.

Where the Prior Enters

The physical part decides the default behaviour:

- which modes exist
- their approximate frequencies
- their approximate damping
- how excitation enters
- how the readout is formed

The neural residual only needs to explain the mismatch:

target — physical model

That is usually easier than learning the whole system from audio.

If the Acceleration Is Neural

Instead of changing the solver, change the acceleration law:

$$\mathbf{a}^n \longrightarrow \mathbf{a}_\theta(\mathbf{q}^n, f^n)$$

For example:

$$\ddot{\mathbf{q}} = \mathbf{a}_\theta(\mathbf{q}, f(t))$$

The update structure stays the same: Störmer-Verlet / velocity Verlet still provides the recurrence on the audio grid.

The learnable object lives inside the acceleration term, not in a new discrete update map.

Fixed-Step Verlet Pattern

```
1 def acceleration(params, q, force):
2     return a_theta(params, q, force)
3
4
5 def verlet_step(params, state, force_n, force_np1, dt):
6     q, v = state
7     a_n = acceleration(params, q, force_n)
8     v_half = v + 0.5 * dt * a_n
9     q_next = q + dt * v_half
10    a_next = acceleration(params, q_next, force_np1)
11    v_next = v_half + 0.5 * dt * a_next
12    return (q_next, v_next)
```

This is the same solver block as before; only `acceleration` is learned.

Scanning Through Time

```
1 def step(state, inputs):
2     force_n, force_np1 = inputs
3     state_next = verlet_step(params, state, force_n, force_np1, dt)
4     y_next = readout(state_next)
5     return state_next, y_next
6
7
8 state_final, y_hat = jax.lax.scan(
9     step,
10    init=(q0, v0),
11    xs=(force[:-1], force[1:]),
12 )
```

Unrolled over time, the solver is still a differentiable recurrence.

Adaptive Solvers

Many Neural ODE papers use adaptive solvers.

They choose internal step sizes based on an error estimate:

Δt changes during the solve

Advantages:

- fewer steps when the dynamics are slow
- more steps when the dynamics are fast
- separates internal solver steps from requested output samples

For audio-rate synthesis, adaptive solvers are not needed: we still need every output sample.

Audio-Rate Constraint

At 44.1 kHz, one second of audio has:

44100

output samples.

Even if the solver uses internal steps, the model still has to produce the full output trajectory:

$$\hat{y}_0, \hat{y}_1, \dots, \hat{y}_N$$

If the solver evaluates the neural acceleration m times per output sample:

$$m \times 44100$$

network evaluations are needed for one second.

For training, this is expensive. This is why we train on short slices of audio.

Training Neural ODEs

State Loss in the Notebook

In `learn_modal_nonlinearity.py`, the target is simulated, so we know the modal state:

$$\mathbf{h}^n = \begin{bmatrix} \mathbf{q}^n \\ \mathbf{v}^n \end{bmatrix}$$

The notebook compares normalized displacement and velocity:

$$\ell_{\text{state}} = \text{mean} \left[\left(\frac{\hat{\mathbf{q}} - \mathbf{q}}{s_q} \right)^2 \right] + \text{mean} \left[\left(\frac{\hat{\mathbf{v}} - \mathbf{v}}{s_v} \right)^2 \right]$$

This is not an audio-only loss: the audio readout is mainly for listening and checks.

With real recordings, we usually do **not** get $\mathbf{q}$ and $\mathbf{v}$.

Then the loss must move to the readout:

$$\mathbf{y}^n = \mathbf{C}\mathbf{h}^n$$

or to a waveform / spectral objective.

Audio Loss

With an audio recording:

$$\hat{y}^n = \mathbf{C}\hat{\mathbf{h}}^n$$

and:

$$\ell_{\text{audio}} = \ell(\hat{y}, y)$$

The gradient must pass through:

$$\text{loss} \rightarrow \text{readout} \rightarrow \text{solver} \rightarrow \mathbf{F}_{\theta}$$

This is powerful, but much harder than fitting observed trajectories.

One-Step Derivative Loss

If clean trajectories are available, we can estimate derivatives:

$$\dot{\mathbf{h}}^n \approx \frac{\mathbf{h}^{n+1} - \mathbf{h}^n}{\Delta t}$$

Then train the vector field directly:

$$\ell_{\text{deriv}} = \frac{1}{N} \sum_n \left\| \mathbf{F}_{\theta}(\mathbf{h}^n, t^n, f^n) - \dot{\mathbf{h}}^n \right\|^2$$

This is simpler than backpropagating through a long rollout.
But finite-difference derivative targets can be noisy.

Windowed Rollout Loss

The notebook does not backpropagate through the full one-second clip on every update. Instead it samples random windows:

$$s_k \sim \text{Uniform}\{0, \dots, N - L\}$$

with:

$$L = 10 \text{ ms} = 441 \text{ samples}, \quad K = 8 \text{ windows}$$

Each window starts from the target state:

$$\hat{\mathbf{q}}_{s_k:s_k+L}, \hat{\mathbf{v}}_{s_k:s_k+L} = \text{Solve}(\mathbf{q}_{s_k}, \mathbf{v}_{s_k}, \mathbf{f}_{s_k:s_k+L})$$

Then we average the normalized state error over windows:

$$\ell_{\text{win}} = \frac{1}{K} \sum_{k=1}^K \ell_{\text{state}}^{(k)}$$

Backpropagating Through Time

For a fixed-step solver, the most direct approach is:

differentiate through every solver step

This is what JAX does when the solve is written with differentiable operations.

Pros:

- simple to understand
- matches the implemented discrete solver
- works well for moderate sequence lengths

Cons:

- memory grows with trajectory length
- gradients can explode, vanish, or become misleading
- long audio signals make the graph very large

The Continuous Adjoint Idea

Neural ODEs are often associated with the adjoint method.

Define the adjoint:

$$\mathbf{a}(t) = \frac{\partial J}{\partial \mathbf{h}(t)}$$

It evolves backward in time:

$$\frac{d\mathbf{a}}{dt} = -\mathbf{a}^\top \frac{\partial \mathbf{F}_\theta}{\partial \mathbf{h}}$$

The gradient with respect to θ is accumulated during this backward solve.
This can reduce memory, but introduces another numerical solve.

Adjoint Caveat

The continuous adjoint differentiates the continuous problem.

The program we run differentiates a **discretised** problem.

These are not always identical:

$$\nabla (\text{discretise then optimise}) \neq \nabla (\text{optimise continuous system})$$

For audio and stiff oscillatory systems, the numerical details matter.

Use adjoints when they help, but do not treat them as magic.

Hybrid Physical-Neural Sound Models

Learn the Missing Force

Recall the physical acceleration we already know:

$$\ddot{\mathbf{q}} = -2\Gamma\dot{\mathbf{q}} - \Omega^2\mathbf{q} + \mathbf{b}f(t)$$

If the model is missing nonlinear physics, the true acceleration may be:

$$\ddot{\mathbf{q}} = -2\Gamma\dot{\mathbf{q}} - \Omega^2\mathbf{q} + \mathbf{b}f(t) + \mathbf{f}_{\text{nl,true}}(\mathbf{q}, \dot{\mathbf{q}}, f(t))$$

A hybrid model keeps the known acceleration and learns the extra force:

$$\ddot{\mathbf{q}} = -2\Gamma\dot{\mathbf{q}} - \Omega^2\mathbf{q} + \mathbf{b}f(t) + \mathbf{f}_{\text{nl},\theta}(\mathbf{q}, \dot{\mathbf{q}}, f(t))$$

The neural term represents the missing nonlinear force, not the whole dynamics.

Neural Force in the State Equation

Write the state equation without introducing a new symbol:

$$\frac{d}{dt} \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ -\Omega^2 & -2\Gamma \end{bmatrix} \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \mathbf{b} \end{bmatrix} f(t) + \begin{bmatrix} \mathbf{0} \\ \mathbf{f}_{\text{nl},\theta}(\mathbf{q}, \dot{\mathbf{q}}, f(t)) \end{bmatrix}$$

The network is forced to behave like an additional force term from unresolved nonlinearities.

Structure Matters

A completely free network can learn nonphysical behaviour:

- energy creation
- unstable damping
- impossible coupling
- dependence on unobserved variables
- poor extrapolation outside the training range

Physical structure reduces the space of bad solutions.

It does not remove the need for checks.

Stable Parameterisations

Common tactics:

- learn damping through a positive map such as `softplus`
- multiply residuals by a small scale at initialisation
- constrain learned matrices to be dissipative
- use bounded activations for force residuals
- penalise energy growth in the loss
- start from short rollouts, then increase horizon

The same theme as Week 4:

differentiable $\neq$ automatically stable

Mini Task: Learn a Missing Residual

Target:

- simulate a modal oscillator with a known nonlinear force
- hide the nonlinear force from the model
- keep the linear modal backbone
- train a small residual network to match the target trajectory

Compare three cases:

linear	linear + learned residual	true nonlinear
--------	---------------------------	----------------

Listen, plot trajectories, and inspect the learned residual.

Notebook Training Choice

Linear stage

- train constrained modal parameters
- no neural residual yet
- objective: same random-window state loss
- result: best linear approximation to nonlinear target

Joint stage

- initialise from the linear fit
- train modal parameters and residual MLP together
- objective: same random-window state loss
- result: physical backbone plus learned restoring-force correction

Derivative matching is an alternative, but not the path used in this notebook.

What to Inspect

Do not trust the scalar loss alone.

Inspect:

- random-window loss curve
- full-second open-loop readout
- readout residual
- first modal trajectories
- learned residual against the hidden-force reference
- exported audio for linear, target, and joint models

The last step is important: a low window loss can still sound wrong in a full rollout.

Practical Problems

Stiffness

Audio physical models are often stiff:

fast high-frequency modes and slow low-frequency decay

Stiff systems force explicit solvers to use very small time steps.

In modal synthesis, the high modes may dominate the numerical step-size constraint even when they are perceptually quiet.

Truncation, damping, and solver choice interact.

Solver Choice Is a Modelling Choice

For this course, ask:

- Does the solver preserve the qualitative behaviour?
- Does it remain stable at the audio sample rate?
- Is it differentiable in the way we need?
- Is it fast enough for training?
- Is it fast enough for synthesis?

The best solver mathematically may be too expensive for the musical task.

Sampling and Observation

The ODE state is continuous-time.

Audio is sampled:

$$\hat{y}^n = \mathbf{C}\mathbf{h}(n\Delta t)$$

Training only sees the sampled output.

If two different trajectories produce similar samples or spectra, the inverse problem may not identify the true dynamics.

State Observability

From a single audio channel:

$$y^n = \mathbf{c}^\top \mathbf{q}^n$$

many modal states can produce similar outputs.

Learning a vector field from audio alone is therefore underconstrained.

Helpful additions:

- known modal basis
- multiple readout positions
- known excitation
- state measurements from simulation
- physics-informed penalties

Generalisation

A useful synthesiser should work beyond the training clip.

Test:

- new initial amplitudes
- new excitation strengths
- longer rollouts
- unseen pitch or parameter settings
- different sampling rates, if the model claims continuous-time behaviour

Continuous-time notation does not guarantee continuous-time generalisation.

Looking Toward PINNs

Neural ODEs vs. PINNs

Neural ODE

Learn or correct the vector field:

$$\dot{\mathbf{h}} = \mathbf{F}_{\theta}(\mathbf{h}, t, f(t))$$

The solver enforces the dynamics during rollout.

PINN

Learn a function and penalise equation residuals:

$$\mathcal{R}_{\theta} = \partial_t u_{\theta} - \mathcal{L}u_{\theta}$$

The loss encourages the solution to satisfy the PDE.

The Bridge

Neural ODEs ask:

What vector field generated this trajectory?

PINNs ask:

What function satisfies this differential equation and data?

Both use automatic differentiation.

They put the physics in different places.

Week 5 Takeaway

Neural ODEs make the dynamics learnable:

$$\dot{\mathbf{h}} = \mathbf{F}_{\theta}(\mathbf{h}, t, f(t))$$

For physical audio, the most useful form is often:

$$\dot{\mathbf{h}} = \mathbf{F}_{\text{phys}}(\mathbf{h}, t, f(t); \phi) + \mathbf{f}_{\text{nl}, \theta}(\mathbf{h}, t, f(t))$$

- keep the physics you trust
- learn the part you do not know
- treat the solver as part of the model
- inspect stability, energy, and generalisation
- do not expect audio loss alone to identify the true dynamics

Resources

Neural ODEs

- Chen, Rubanova, Bettencourt, Duvenaud, “Neural Ordinary Differential Equations”, 2018.
- Kidger, *On Neural Differential Equations*, 2022.
- Kidger, “[DiffraX](#): numerical differential equation solvers in JAX”.

Hybrid physical-neural dynamics

- Rackauckas et al., “[Universal Differential Equations for Scientific Machine Learning](#)”, 2020.
- Greydanus, Dzamba, Yosinski, “Hamiltonian Neural Networks”, 2019.
- Cranmer et al., “[Lagrangian Neural Networks](#)”, 2020.